

10 — Lab Cheat Sheet

DFIR Study Notes — exam cards from classroom labs (detection / forensics focus). Full-resolution screenshots also appear on the HTML page.

Scope: Tool and command strings are IOC / hunt patterns — not attack how-tos. No lab zip/HTML packs are included.

Sections: A Static/PE · B Triage · C Cred/DCSync/IR · D Persistence · E Volatility · F NTFS

A. Static / sandbox commands & PE

01 — What each command does

WHAT EACH COMMAND DOES

- `mkdir -p output` : creates an `output/` sub-folder in the lab directory for your analysis results; `-p` suppresses an error if it already exists.
- `sha256sum ... | tee ...` : computes the **SHA-256** of the sample (your primary forensic fingerprint) and, via `tee`, both prints it to screen *and* saves it to a file.
- `file ...` : identifies the true file type by reading the binary header, regardless of the filename extension. This proves whether it really is a PE executable.
- `ls -lh ...` : shows the size in human-readable form (`-h`), plus permissions and timestamp. An unexpectedly large or small file is itself a clue.

WHAT EACH COMMAND DOES — `mkdir`, `sha256sum|tee`, `file`, `ls -lh`

02 — The report's static panel

THE REPORT'S STATIC PANEL

Your workstation checks (hash, type, size, strings) are the disk-side view. When the report comes back, its **File Details / Static Analysis** section parses the same PE more deeply. Read it before the behaviour:

- **Section entropy.** Entropy scores randomness from 0–8.0. A section above **~7.0** (approaching 7.9) is compressed or encrypted, the rigorous signature of **packing**. A *raw size* far smaller than the *virtual size* confirms it: the section expands in memory at run time.
- **imphash.** A hash of the PE's *import table*. Unlike the SHA-256 (which changes on a one-byte edit), the imphash clusters variants built from the same code into one family.
- **ssdeep (fuzzy hash).** Measures how *similar* two files are, grouping near-identical samples an exact hash would treat as unrelated.
- **Imports = capability.** The OS functions a PE intends to call hint at what it does before it runs:
`VirtualAlloc` + `WriteProcessMemory` + `CreateRemoteThread` together advertise **process injection**.
An implausibly *thin* import table is itself a packing tell.

Record the **imphash** alongside the SHA-256, so related variants group into one family instead of arriving as unrelated alerts.

THE REPORT'S STATIC PANEL — entropy/packing, imphash, ssdeep, imports

03 — PE/MZ + process-tree lineage

PE / MZ

PE (Portable Executable) is the standard file format for Windows executables, DLLs, and drivers. **MZ** refers to the two magic bytes `4D 5A` ("MZ") at the very start of every PE file, named after Mark Zbikowski, an early MS-DOS architect. So when the report says a dropped file "carries an MZ/PE header," it means the malware wrote another Windows executable to disk.

PROCESS-TREE LINEAGE

The **process tree** records which process started which. Malicious activity often announces itself through an anomalous parent-child relationship with no legitimate explanation. The canonical example to memorise: opening a Word document runs `WINWORD.EXE`, and that application has *no* reason to launch a shell, so `WINWORD.EXE → cmd.exe` or `→ powershell.exe` almost always means a malicious macro is now running commands. Many of the most reliable EDR rules are expressed as exactly this lineage, and they are hard to evade because the technique *requires* that parentage.

Our sample is a **standalone dropper** (no Office parent here), but the skill is the same: open each process for its **command line** (where an encoded PowerShell payload becomes visible) and its **injection targets**, then locate the **persistence**: a service, WMI subscription, or Run key, because that is both the detection and the cure.

PE/MZ + PROCESS-TREE LINEAGE (WINWORD→cmd/powershell)

04 — MITRE ATT&CK Day-1 subset

MITRE ATT&CK

The report's own **ATT&CK Techniques Detection** panel summarises the run as **246 indicators mapped to 102 attack techniques across 10 tactics** - a different, broader tally than the 8 · 55 · 200 indicator counts on the summary card. A Day-1 subset worth knowing:

- **T1566.001** Phishing: Spearphishing Attachment · **T1204.002** User Execution: Malicious File (the macro)
- **T1137** Office Application Startup · **T1059.001** PowerShell · **T1059.003** Windows Command Shell
- **T1027** Obfuscated Files or Information (base64 `-EncodedCommand`) · **T1105** Ingress Tool Transfer · **T1071.001** Web Protocols
- **T1543.003** Windows Service · **T1053.005** Scheduled Task

MITRE ATT&CK Day-1 subset (T1566.001 ... T1053.005)

B. Triage & artifacts

05 — Triage = selective copy

A triage collection is a **selective copy**: a folder tree of individually collected files you browse directly - there is no volume to mount, and no unallocated space, file slack or deleted-file recovery, all of which a full disk image would give you at the cost of hours of acquisition and hundreds of gigabytes. The manifest files matter because the package **documents itself**: `collection_context.json` records what was asked for and when, `uploads.json` lists exactly which files were captured with their hashes and sizes, and `log.json` records errors and skips. Together they let you prove what is in the evidence - and what was never collected - without relying on the analyst's memory.

Triage = selective copy; collection_context / uploads / log.json

06 — Artifact category → path

Artifact category	Location
Registry hives	<code>Windows\System32\config\</code>
Event logs	<code>Windows\System32\winevt\Logs\</code>
File-system metadata (<code>\$MFT</code> / <code>\$J</code>)	volume root / <code>\$Extend</code>
Prefetch	<code>Windows\Prefetch\</code>
WMI repository	<code>Windows\System32\wbem\Repository\</code>
Scheduled tasks	<code>Windows\System32\Tasks\</code>
BITS jobs database	<code>ProgramData\Microsoft\Network\Downloader\</code>
Amcache	<code>Windows\AppCompat\Programs\Amcache.hve</code>

Artifact category → path (Registry, EVTX, MFT, Prefetch, WMI, Tasks, BITS, Amcache)

07 — Dirty SOFTWARE hive

Registry Explorer flagged the collected `SOFTWARE` hive as **dirty**.

a. Tick what "dirty" means here. (*tick one*)

- ☐ Windows was live when the hive was copied, so its transaction logs still hold uncommitted changes
- ☐ The hive was corrupted by the collection tool
- ☐ The hive contains malware

Dirty SOFTWARE hive quiz — live copy + transaction logs

C. Cred dump / DCSync / IR response

Detection framing: Credential-access tooling is listed only as what Security/Sysmon show and what to hunt. Treat dump-tool command lines as **IOC strings** (process name + path + target), not procedures to run.

08 — DCSync vs legitimate replication

Directory replication is a **domain-controller-to-domain-controller** protocol: legitimate requests come from machine accounts ending in `$`, on a schedule. An interactive administrator account asking for `DS-Replication-Get-Changes-All` in a two-minute burst is not replication - it is a tool run. What it yields is categorically bigger than the LSASS dump: the dump exposed only the secrets cached on *this* host, whereas DCSync returns the password hash of **every account in the domain** - including `krbtgt`, which enables forged Kerberos golden tickets - without the attacker ever touching another machine.

DCSync vs legitimate DC↔DC replication; krbtgt risk

09 — No 4104 when dump is a native EXE

No 4104 script block wraps the dump. Script Block Logging is on and the timeline holds 65 script blocks from this host, but not one of them mentions `lsass`, `procdump` or a `.dmp` path - none resolves the LSASS PID or launches the dumping tool. Look back at the command line: `procdump64.exe -accepteula -ma 748 C:\Users\Public\lsass.dmp` carries a **hard-coded PID** and was executed *directly*, not through a PowerShell script that discovered the PID first. Script Block Logging only records PowerShell script text, so a direct executable launch leaves it untouched. The absence is a finding, not a gap in your search - and it is why **Security 4688 with command-line logging is the main detection source** for this incident.

No 4104 when dump is native EXE; 4688 cmdline is the primary detection surface (hunt patterns such as dump-tool name + LSASS target — not a how-to)

10 — DCSync impact & remediation

Because DCSync returns the directory itself, the impact covers **every account in the domain**, not the handful cached on this host - so account-by-account triage is the wrong model. In particular `krbtgt` is exposed: its hash signs all Kerberos tickets, so an attacker can mint **golden tickets** impersonating any user, including ones created after the incident. Removing this attacker's access therefore means a **domain-wide credential reset**, with `krbtgt` reset *twice* (allowing for replication between resets), not just rebuilding host `AD`.

DCSync impact: domain-wide reset; krbtgt twice

11 — Domain compromise IR

Treat this as a **domain compromise, not a host compromise**. Immediately: isolate host `AD` while preserving it as evidence; disable or reset the `Administrator` account used for the dump and the replication; then plan a **domain-wide credential reset** - all privileged and service accounts first, with `krbtgt` reset **twice** to invalidate forged tickets. In parallel, hunt for the attacker's return path before resetting anything they could watch (the persistence mechanisms in Lab 2-3), restrict directory-replication rights to DC machine accounts only, and deploy the two detections above so a repeat is caught in minutes rather than after the fact.

Domain compromise IR: isolate AD, hunt return path, krbtgt×2

D. Persistence

12 — Layered persistence

Categories beyond a Startup-folder shortcut include: **registry autostart values** (`Run` / `RunOnce`, `Winlogon` shell/userinit); **scheduled tasks**; **Windows services**; **permanent WMI event subscriptions**; **BITS jobs**; and **hijack-based** techniques such as DLL search-order or COM hijacking. An operator layers several because they are *independent* and live in different subsystems: each is listed by a different tool, so a defender who finds one and stops looking leaves the others intact. Redundancy also makes removal harder - if one mechanism re-fetches the payload, deleting the file alone achieves nothing. For the defender the lesson is that remediation requires a **complete list first**, then removal in one coordinated action.

Layered persistence categories; complete list then remove

13 — Mutually restoring persistence

The four mechanisms are independent and **mutually restoring**: the BITS job re-downloads the payload, and any surviving launcher runs it again, so a partial clean-up simply puts you back to the start. Removing them one at a time also **warns the attacker** - they still hold valid credentials (Lab 2-2), so a partial remediation invites them to re-establish before you finish. The discipline is therefore: *list everything first, remove it all inside one coordinated window, then verify* - and treat the stolen credentials as part of the same remediation, not a separate task.

Mutually restoring persistence; coordinated removal window

E. Memory forensics / Volatility

14 — Boot time + ProfileList vs sessions

windows.info does **not** report boot time. System (**PID 4**) create time $\approx$ boot time.

```
vol -f <memory.raw> windows.info
vol -f <memory.raw> windows.pslist
```

Plugin / key	Answers
ProfileList (HKLM\...\ProfileList)	Every account that has ever logged in
windows.sessions	Who was logged in at capture

```
vol -f <memory.raw> windows.registry.hivelist
vol -f <memory.raw> windows.registry.printkey \
--key "Microsoft\Windows NT\CurrentVersion\ProfileList"
vol -f <memory.raw> windows.sessions
vol -f <memory.raw> windows.pstree
```

15 — pslist vs psscan vs pstree

Plugin	What it does
windows.pslist	Walks the active process list (OS view).
windows.psscan	Scans raw memory for process structs — finds exited / unlinked.
windows.pstree	Parent/child tree — spot impossible lineages.

Exam tip: pslist $\neq$ psscan $\rightarrow$ hidden/exited (DKOM unlink). Use pstree for weird parents.

16 — cmdline, dlllist, getsids, handles

Plugin	Shows	Why it matters
cmdline	Full launch command line	Encoded/obfuscated args = strong indicator
dlllist	Loaded modules/DLLs	Networking DLL in non-network process

getsids	Process SIDs / authority	Unprivileged app as SYSTEM; S-1-5-18 = SYSTEM
handles	Files, keys, named pipes	Pipes = local malware IPC

17 — netscan + malfind

windows.netscan: TCP/UDP endpoints (incl. closed); local/remote, state, owning PID.

windows.malware.malfind: executable+writable, no file backing — injected/reflective code; hexdump for MZ (4D 5A).

JIT runtimes (browsers, .NET) cause RWX FPs. Interesting = RWX **and** MZ **and** odd process. Treat malfind as **leads**, not conclusions.

18 — modules vs driverscan

Plugin	Walks	Named as
windows.modules	Loaded module images	Files: ntoskrnl.exe, paths under \SystemRoot\system32\
windows.driverscan	Driver objects in memory	Device ns: \Driver\HTTP, \Driver\00000311

Counts/name lists do **not** prove anything (different namespaces). Map by **address**: driverscan Start inside modules Base...Base+Size. Start in no module = investigate.

19 — What modules/driverscan showed

What it showed: the kernel module list and the driver-object table were both listed and recorded, 141 and 117 records, and neither contains anything that stands out when you look at it. That is a documented look at kernel space, and it belongs in the report as exactly that.

What it did not show: that the kernel is clean. Running these two plugins only *lists* what each one sees. It is not a rootkit test. A real test maps each driver object's **Start** address into a loaded module's **Base .. Base + Size** range and looks for an object that belongs to no loaded image - that is beyond today's scope, and without it you cannot say the kernel was verified.

What modules/driverscan showed vs did not (listing ≠ clean)

20 — vadinfo: hollowing vs reflective

Two columns do the work. windows.vadinfo separates file-backed vs private code. It does **not** separate hollowing from reflective — both are private. Also check whether the main image region is intact. Read both before naming a technique.

21 — Injection vs hollowing vs reflective

Technique	What the attacker does	How it looks in memory
DLL injection	Forces a legitimate process to load a malicious <code>.dll</code> from disk.	An unexpected module in <code>dlllist</code> , backed by a file - often from a non-standard path such as <code>%TEMP%</code> . The DLL exists on disk, so disk forensics can find it too.
Process hollowing	Starts a legitimate program suspended, unmaps its real code, writes malicious code into the same address space, and resumes it.	The process keeps a trusted name and path , but its main image region is private rather than file-backed , or its in-memory contents do not match the file on disk. Often a suspiciously <i>short</i> module list and no user parent.
Reflective loading	Maps a complete executable into memory itself, bypassing the Windows loader entirely. Nothing is written to disk.	A private RWX region containing an MZ header , belonging to no module in <code>dlllist</code> at all - the executable is invisible to the loader, so it appears in no module list.

None of the three is the same as a DLL being invoked on purpose. Each row above describes code arriving in a process that did not ask for it. On this host you will also meet a DLL that was loaded because a program was *explicitly told* to load it, on its command line. That is not injection, and it is not side-loading either: **side-loading** means planting a DLL where a legitimate program will pick it up *by name*, through its own search order, without being told. Keep the two apart when you report, because they need different detections and they say different things about the attacker.

DLL injection vs hollowing vs reflective; ≠ side-load

22 — malfind dump vs dumpfiles

Different technique, different recovery method. The Exercise 4 implant lived in a private memory region, so `malfind --dump` carved it out. This component is a real file that was dropped on disk and started, so `malfind` has nothing to give you and `windows.dumpfiles` is the plugin that does.

Private region → `malfind --dump`; on-disk → `dumpfiles` (forensic extraction)

F. NTFS / MFTECmd

23 — NTFS 0x10 vs 0x30 (timestamp lead)

Every NTFS file carries two sets of timestamps. The `0x10` set is what Windows shows in Explorer, and a program can rewrite it. The `0x30` set is written by the kernel when a file is created, renamed or moved, and ordinary tools cannot set it. Changing the `0x10` set to hide when a file arrived is called **timestamping**, and it leaves a pattern: a `0x10` created time earlier than the `0x30` one.

Copying a file can leave that same pattern, because a copy can inherit the metadata of its source. So treat the pattern as a reason to look, not as a finding on its own. The arrival time you can defend is the `0x30` value, checked against the request and process records you already have.

NTFS 0x10 vs 0x30; timestamp pattern is a lead

24 — MFTECmd

Eric Zimmerman MFT parser — \$MFT / \$J (USN) timelines, create/modify/delete, 0x10 vs 0x30.

`C:\Training\Tools\EricZimmerman\MFTECmd.exe`

Pairs with: Timeline Explorer, AmcacheParser, Hayabusa.

Quick exam anchors

- Static order: hash → file → entropy/imphash → imports → sandbox lineage
- Dirty hive → collect hive and transaction logs
- Native dump EXE → no PS 4104; hunt 4688 / Sysmon 1 cmdline IOCs
- DCSync remediation → domain-wide reset + krbtgt ×2
- pslist ≠ psscan; malfind = leads (RWX+MZ+odd process)
- modules vs driverscan → map by address, not name/count
- vadinfo: private vs file-backed; both hollow + reflective are private
- Timestamp lead: SI (0x10) vs FN (0x30) mismatch

DFIR Study Notes — detection focus
Educational / defensive material only. No attack how-tos.